Binary Arithmetics and Complements

Basic Binary Arithmetic

Binary mathematics forms the foundation of digital systems. Subtraction relies on a borrow mechanism, which is especially important when subtracting 1 from 0. Multiplication follows the exact same logical behavior as an AND gate—the product is only 1 if both the multiplier and multiplicand are 1.

Addition Rules

Binary addition works exactly like decimal addition, but instead of carrying over when you reach 10, you carry over when you reach 2 (which is `10` in binary). The most important rule to remember is what happens when you add three 1s together (which often happens when you have a carry bit from a previous column).

$$
\begin{aligned}
0 + 0 &= 0 \\
0 + 1 &= 1 \\
1 + 0 &= 1 \\
1 + 1 &= 10 \quad (\text{Sum of } 0, \text{ Carry of } 1) \\
1 + 1 + 1 &= 11 \quad (\text{Sum of } 1, \text{ Carry of } 1)
\end{aligned}
$$

Subtraction Rules

Binary subtraction relies on a "borrow" mechanism, similar to decimal subtraction. The tricky part is when you need to subtract 1 from 0. You must borrow from the next highest-order bit. When you borrow 1 from the next column to the left, it acts as a `10` (which is 2 in decimal) in your current column.

$$
\begin{aligned}
0 - 0 &= 0 \\
0 - 1 &= 1 \ (\text{with a borrow of } 1) \\
1 - 0 &= 1 \\
1 - 1 &= 0
\end{aligned}
$$

Multiplication Rules

Multiplication in binary is simpler than in decimal because you are only ever multiplying by 0 or 1. It perfectly mirrors the logic of an AND gate: the product is only 1 if both the multiplier and multiplicand are 1. The process involves shifting partial products to the left and adding them together.

$$
\begin{aligned}
0 \times 0 &= 0 \\
0 \times 1 &= 0 \\
1 \times 0 &= 0 \\
1 \times 1 &= 1
\end{aligned}
$$

Binary Division

Binary long division follows the exact same algorithm as decimal long division: Divide, Multiply, Subtract, Bring down.

1. See if the divisor is less than or equal to the current portion of the dividend.
2. If it is, place a `1` in the quotient, subtract the divisor from that portion, and bring down the next bit.
3. If it is not, place a `0` in the quotient, bring down the next bit, and try again.
   This process heavily relies on your mastery of binary subtraction.

   Example 1: 1001 ÷ 11

$$\begin{array}{r} 11 \quad \text{(Quotient)} \\ \hline 11) \quad 1001 \\ \quad -11 \downarrow \\ \hline \quad 011 \\ \quad -\ 11 \\ \hline \quad 0 \quad \text{(Remainder)} \end{array}$$

   Example 2: 101101 ÷ 110

$$\begin{array}{r} 111 \quad \text{(Quotient)} \\ \hline 110) \quad 101101 \\ \quad -110 \downarrow\downarrow \\ \hline \quad 1010 \\ \quad -110 \downarrow \\ \hline \quad 1001 \\ \quad -\ 110 \\ \hline \quad 011 \quad \text{(Remainder)} \end{array}$$

Signed Numbers & Complements

Because Arithmetic Logic Units (ALUs) are designed primarily to add, computer architecture represents negative numbers using a method called 2's complement. This allows systems to perform subtraction simply by adding a negative number. The Most Significant Bit (MSB)—the leftmost bit—acts as the sign indicator: `0` means positive, and `1` means negative.

Theory

To convert a positive binary number to its negative counterpart (2's complement), you follow a two-step algorithm: First, find the 1's complement by flipping all bits (0s become 1s, and 1s become 0s). Second, add 1 to the least significant bit of the result.

Finding -7 from +7 in 5-bit representation

$$\begin{aligned} +7 &= 00111 \\ \text{1s Complement} &= 11000 \\ \text{2s Complement } (-7) &= 11000 + 1 = 11001 \end{aligned}$$

Theory

This complement logic is universal and applies to other base systems, such as Octal (Base-8). To find the 8's complement of an octal number, you first calculate the 7's

complement by subtracting every single digit from 7, and then you add 1 to the final result.

8's Complement of 7263

$$\begin{aligned} 7\text{s Complement} &= 7777 - 7263 = 0514 \\ 8\text{s Complement} &= 0514 + 1 = 0515 \end{aligned}$$

2's Complement Arithmetic

When performing addition with signed binary numbers of a fixed bit length (such as a 5-bit system), any extra "carry" bit that gets generated beyond that fixed length is strictly discarded. The remaining MSB will correctly dictate whether the final answer is positive or negative.

Case 1: Adding two negative numbers. When we add -7 and -4, an overflow bit is generated. We discard it, and the remaining 5 bits give us the correct negative result.

-7 + -4 = -11

$$\begin{gathered} 11001 + 11100 = \ 1 \ \ 10101 \\ \text{Discard the 6th bit.} \quad \text{MSB is 1, resulting in } -11. \end{gathered}$$

Case 2: Adding a positive and a negative number where the positive is larger in magnitude. A 6th carry bit is generated and discarded. The MSB of the remaining 5 bits is 0, indicating a positive result.

+7 + -4 = +3

$$\begin{gathered} 00111 + 11100 = \ 1 \ \ 00011 \\ \text{Discard the 6th bit.} \quad \text{Result is eactly } +3. \end{gathered}$$

Case 3: Adding a positive and a negative number where the negative is larger in magnitude. In this scenario, no 6th carry bit is generated. The MSB naturally resolves to 1, indicating the result is a negative number.

+4 + -7 = -3

$$\begin{gathered} 00100 + 11001 = 11101 \\ \text{No 6th bit generated.} \quad \text{MSB is 1, resulting in } -3. \end{gathered}$$